

Chat Application

Real-Time Messaging with Secure Authentication

Project Report



Document	Project Report
Version	1.0
Date	August 2026
Stack	Android (Kotlin) + Node.js / Socket.io
Deliverables	Source code, README, APK, this report

Abstract

This project presents a real-time messaging application that enables users to exchange text and media instantly over a WebSocket connection while protecting accounts with token-based authentication. The system is split into a native Android client written in Kotlin and a lightweight Node.js backend that combines a REST API (authentication, contacts, chat history, media upload) with a Socket.io server for live message delivery, presence, typing indicators and read receipts. Users register or log in with an email and password that is hashed using bcrypt, receive a signed JWT, and then communicate with any other registered user in real time. Messages are persisted on the server so full chat history can be replayed at any time, and a foreground service delivers push-style notifications when new messages arrive while the app is in the background. The deliverables include the complete source code, this report, a public repository, and a ready-to-install APK.

1. Introduction

Instant messaging is one of the most widely used forms of digital communication. Modern chat applications are expected to deliver messages with low latency, show whether contacts are online, support photo sharing, keep a searchable history, and notify users when they are not actively looking at the screen. This project implements all of these capabilities with a clean, layered architecture that is easy to run locally and extend in the future.

2. Objectives

1. Implement secure user registration and login with password hashing and JWT sessions.
2. Provide real-time, low-latency message exchange using Socket.io over WebSockets.
3. Broadcast live online/offline status across all connected clients.
4. Send push-style notifications for messages received while the app is backgrounded.
5. Allow image sharing through a picker, upload endpoint and inline rendering.
6. Persist messages on the server and cache them locally for full chat history.
7. Ship a documented codebase, a prebuilt APK and a project report.

The application is also designed to demonstrate end-to-end networking on a single machine: two Android emulator instances (or an emulator and a physical device) can chat through one local Node.js server.

3. Technology Stack

Layer	Technology	Purpose
Android client	Kotlin	Primary app language
UI	Jetpack / Material 3 / ViewBinding	Screens, components, typed view access
Networking (REST)	Retrofit + OkHttp + Gson	Auth, contacts, history, media upload
Real-time layer	Socket.io Java client	Live messages, presence, typing, receipts
Images	Coil	Async image loading in chat bubbles
Backend	Node.js + Express	REST endpoints and static media

Realtime server	Socket.io (Node)	Bidirectional event transport
Auth	jsonwebtoken + bcryptjs	JWT sessions and password hashing
Uploads	Multer	Multipart image upload handling
Storage	JSON file store + SharedPreferences	Server persistence & local cache

4. System Architecture

The system follows a classic client-server model. The Android app is the only client; the Node.js process hosts both the REST API and the Socket.io engine on the same port. Communication is split into two channels:

REST channel. Used for account creation, login, listing contacts, fetching chat history and uploading images. Every request is authenticated with a JWT sent in the Authorization header.

WebSocket channel. Used for everything real-time: sending/receiving messages, online status broadcasts, typing indicators and read receipts. The client passes its JWT in the socket handshake, which the server validates before the connection is accepted.

Data flow

When user A sends a message, the client emits a `message : send` event. The server validates the payload, persists the message to the JSON store, broadcasts `message : new` to the recipient's room and confirms to the sender with a server-side acknowledgement. Presence changes are propagated with `user : status` events so every client always sees accurate online indicators. Read receipts travel in the reverse direction via `message : read`.

Android client architecture

The Android app is organised into a data layer (Retrofit service, SocketManager singleton, session store and message cache) and a UI layer (Login, Contacts and Chat screens). The SocketManager is a process-wide singleton created on app start; activities register and unregister listeners so the same live connection serves every screen. Chat history is loaded from the server on opening a conversation and mirrored to a local cache so the app still shows recent messages offline.

5. Feature Implementation

5.1 User Authentication

Registration and login are REST calls to `/api/auth/register` and `/api/auth/login`. Passwords are hashed with bcrypt (10 salt rounds) and never stored or transmitted in plain text. A successful request returns a signed JWT (30-day expiry by default) plus the user profile. The token is stored in SharedPreferences, attached to every REST request as a Bearer token, and used to authenticate the socket handshake. Validation covers required fields, email format and minimum password length, with per-field error messages in the UI.

5.2 Real-Time Messaging

Messages are delivered over the persistent WebSocket connection, giving near-instant delivery without HTTP polling. The sender's message appears after the server acknowledges it; incoming messages are pushed to the open conversation in real time. Conversations automatically mark incoming messages as

read and the sender sees a tick/double-tick receipt indicator.

5.3 Online Status

The server tracks the socket id of every connected user. On connect/disconnect it updates the user record and broadcasts a user:status event containing the user id, the online flag and a last-seen timestamp. The contacts screen renders a green dot for online users and refreshes instantly when a user comes online or goes offline; the chat header shows the same live status.

5.4 Push Notifications

A foreground service (ChatConnectionService) keeps the socket alive while the app is in the background. When a message arrives and the app is not visible, the service builds a high-priority notification with a pending intent that opens the correct conversation. Notifications require the POST_NOTIFICATIONS runtime permission on Android 13+ and are requested automatically after login.

5.5 Media Sharing

The attach button opens the system image picker (ActivityResultContracts.GetContent). The chosen image is uploaded as a multipart request to /api/media/upload, which validates type and size (JPEG/PNG/GIF/WebP, max 10 MB) and returns a public URL. The client then sends an image-type message containing that URL; both bubbles render the photo with Coil and tap-through opens the full image.

5.6 Chat History

Every message is persisted by the server in backend/data/messages.json with sender, recipient, type, content, timestamp and read state. The REST endpoint /api/auth/messages/:userId returns the complete thread between two users (latest N messages, capped at 500). The Android client caches threads locally in SharedPreferences, so history is available immediately when reopening a conversation and is then reconciled with the server.

6. API and Event Reference

REST endpoints (all under /api/auth/* require a Bearer token):

Method	Endpoint	Description
POST	/api/auth/register	Create an account, returns {token, user}
POST	/api/auth/login	Authenticate, returns {token, user}
GET	/api/auth/users?q=	List users with live online status
GET	/api/auth/messages/:userId	Full chat history with a user
POST	/api/media/upload	Multipart image upload, returns {url}
GET	/uploads/:filename	Serve uploaded media
GET	/health	Server health check

Key Socket.io events:

Emit	Payload	Receives	Purpose
message:send	{to, type, content/mediaUrl}	message:new, message:ack	Send a message

message:read	{from}	message:read	Read receipts
typing:start / stop	{to}	typing:start / stop	Typing indicator
-	-	user:status	Presence broadcast

7. Security Considerations

- Passwords are hashed with bcrypt; the plaintext is never persisted or logged.
- JWTs are signed with a server-side secret and carry the user id, name and email.
- Socket handshakes are authenticated and rejected when the token is missing, invalid or expired.
- All REST endpoints except register/login validate the Bearer token.
- Uploaded files are restricted to image MIME types and a 10 MB limit; filenames are regenerated to avoid path traversal.
- Cleartext HTTP is enabled for local development only; production deployments should use HTTPS and a strong, environment-provided JWT_SECRET.

8. Testing

The backend was validated end-to-end with an automated script that exercises the full user journey against a running server. The following scenarios passed:

- Registration returns 201 with a token and profile; duplicate email returns 409.
- Login returns 200 with a token; wrong credentials return 401.
- Two socket clients connect with JWT handshake auth.
- Sending a message returns a success acknowledgement to the sender.
- The recipient receives the message in real time over the socket.
- Read receipts propagate back to the sender.
- GET /api/auth/messages/:userId replays the stored chat history.
- Disconnecting a client broadcasts an accurate offline presence event.
- Multipart image upload returns a 201 with a publicly servable URL.

The Android app builds cleanly with Gradle (AGP 8.9.2, Kotlin 2.0.21, compileSdk 36) and produces a debug APK of approximately 7.4 MB.

9. Deliverables

- Public GitHub repository containing the full project.
- Complete source code for the Android client (Kotlin) and Node.js backend.
- README documentation covering setup, configuration and usage.
- Prebuilt APK at apk/ChatApp-v1.0.0-debug.apk.
- This project report (PDF).

10. Conclusion and Future Scope

The Chat Application meets every stated requirement: secure authentication, real-time messaging, online status, push notifications, media sharing and persistent chat history. Its layered design keeps the networking, data and UI concerns separate, which makes the codebase readable and extensible. Because the backend stores data as JSON files and requires no external services, the entire system can be demonstrated on a single machine in minutes.

Future work could replace the JSON file store with a production database (PostgreSQL or MongoDB), add end-to-end encryption for message payloads, integrate Firebase Cloud Messaging for delivery even when the app process is killed, support voice/video calls and group chats, add push-to-refresh pagination for very long threads, and publish the app to Google Play with a release-signed APK.

--- End of report ---